Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 5

Дисциплина МДК 03.02

Разбор кейсов оценки качества программного обеспечения

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Модель и шкала качества.....3

1.2 Разбор реального кейса Zip Slip.....4

1.3 План снижения рисков.....4

1.4 Выводы.....5

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....6

ПРИЛОЖЕНИЕ А Листинг исходного кода.....7

1 ВЫПОЛНЕНИЕ РАБОТЫ

Я выполнил работу по варианту 2 Library System АИС «Умный кампус» на C# и .NET 8. Модуль принимает PDF/EPUB/ZIP в карантин; права Reader и Librarian разделены [3]. Оформление выполнено по положению института [2].

1.1 Модель и шкала качества

Профиль построен по восьми характеристикам ГОСТ Р ИСО/МЭК 25010-2015. В международной редакции ISO/IEC 25010:2023 девять характеристик, поэтому старая схема не названа новой моделью [1] [4].

Я принял рабочую шкалу: 1 — функция отсутствует; 2 — частичная реализация; 3 — основные меры есть, остаются существенные ограничения; 4 — поведение проверено, нужны отдельные улучшения; 5 — подтверждены все критерии в согласованной эксплуатационной среде. Шкала авторская и не означает сертификацию.

В таблице 1 приведена матрица восьми характеристик с оценками и основаниями по каждой из них.

Таблица 1 — Профиль качества

Характеристика	Балл	Подтверждение
Функциональная пригодность	4	PDF/EPUB/ZIP, каталог и карантин
Эффективность производительности	3	Лимиты размеров и частоты
Совместимость	3	HTTP, JSON, multipart, браузер и HttpClient
Удобство использования	4	Явные роли и статусы результата
Надёжность	4	Проверка всего архива до записи, транзакция
Защищённость	3	Роли, пути, сигнатуры, карантин и аудит
Сопровождаемость	4	Разделены политика, сервис и контроллер
Переносимость	3	Стандартное решение NET 8 и SQLite

Источник: собственные результаты автора; характеристики по [1]

1.2 Разбор реального кейса Zip Slip

Я рассмотрел исследование Snyk 2018 года о Zip Slip. Недоверенное имя ZIP-записи с ../ выводит ручную распаковку за каталог назначения. Последствия зависят от прав процесса и перезаписанного файла. Ошибка относится к небезопасному обработчику, а не любой штатной операции .NET [6].

Для библиотеки нарушаются целостность и защищённость; перезапись настроек угрожает надёжности и доступности. В исходном образце риск создают Path.Combine с entry.FullName и запись непроверенных байтов.

1.3 План снижения рисков

Я запретил опасные сегменты, абсолютные пути, ссылки и дубли. Все записи проверяются до сохранения; сервер выбирает UUID имени. Новые книги остаются в приватном карантине, ограничения байтов и частоты снижают риск исчерпания ресурсов. Меры согласованы с рекомендациями OWASP по загрузкам [5].

Сведения представлены в таблице 2.

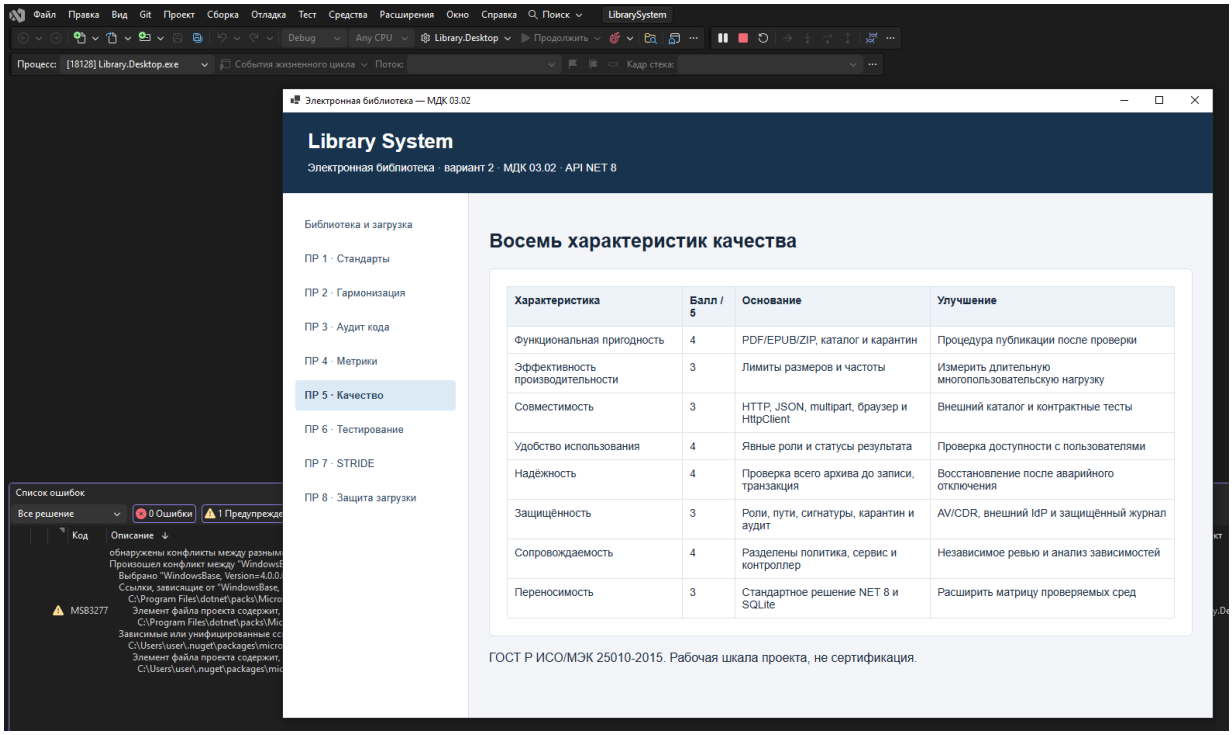
Таблица 2 — Меры и остающиеся улучшения

Риск	Реализовано	Следующий шаг
Перезапись чужого пути	Проверка пути и UUID	Сохранить регрессионные тесты
Вредоносное содержимое	Карантин без выдачи новых файлов	AV/CDR перед публикацией
Отказ из-за нагрузки	Размеры, ZIP и Rate Limiting	Измерить многопользовательскую нагрузку
Изменение следов	Журнал без секретов	Защищённое удалённое хранилище

Источник: составлено автором по результатам работы

Баллы не завышены: корректная сигнатура PDF не доказывает безопасность документа, локальный журнал изменяем администратором, а стенд не подтверждает промышленный SLA. Вкладка «ПР 5 · Качество» показывает профиль и план улучшений.

На рисунке 1 показана матрица восьми характеристик, основания оценок и предложенные улучшения.



Источник: снимок прогона на Windows, предоставленный автором.

Рисунок 1 — Матрица восьми характеристик качества

1.4 Выводы

Я составил профиль с основаниями оценок и связал реальный класс отказов с конкретными характеристиками. Приоритетные меры — контроль архивного пути, карантин и ограничения ресурсов. Дальнейшие улучшения отделены от уже выполненных проверок.

Полные исходники, входные данные и журналы переданы с отчётом. Основной листинг приведён в приложении А.

Исходный код и порядок запуска: репозиторий MDK0302-LibrarySystem.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р ИСО/МЭК 25010-2015. Системная и программная инженерия. Модели качества систем и программных продуктов. — URL: https://protect.gost.ru/gost/details/e5744d86-5a17-4909-96da-b38691326f0c (дата обращения: 08.11.2026) [1].

2. Положение об оформлении письменных работ. Санкт-Петербургский институт экономики и управления. Методические материалы [2].

3. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [3].

4. ISO/IEC 25010:2023. Systems and software engineering. SQuaRE. Product quality model. — URL: https://www.iso.org/standard/78176.html (дата обращения: 08.11.2026) [4].

5. OWASP. File Upload Cheat Sheet. — URL: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (дата обращения: 08.11.2026) [5].

6. Snyk Security Research. Zip Slip Vulnerability. Исследование 2018 года. — URL: https://security.snyk.io/research/zip-slip-vulnerability (дата обращения: 08.11.2026) [6].

ПРИЛОЖЕНИЕ А
Листинг исходного кода

src/Library.Tools/Program.cs

```
using System.Diagnostics;
using System.Text;
using System.Text.Json;
using System.Text.Encodings.Web;
using System.Xml.Linq;
using Library.Tools;
using Library.Core;
Console.OutputEncoding = Encoding.UTF8;
var directory = new DirectoryInfo(Directory.GetCurrentDirectory());
while (directory is not null && !Directory.Exists(Path.Combine(directory.FullName, "artifacts"))) directory =
directory.Parent;
string root = directory?.FullName ?? throw new IOException("Запуск из корня Project");
string verification = Path.Combine(root, "verification");Directory.CreateDirectory(verification);
var options = new JsonSerializerOptions(JsonSerializerDefaults.Web) { WriteIndented = true, Encoder =
JavaScriptEncoder.UnsafeRelaxedJsonEscaping };
void Save(string name, object data) => File.WriteAllText(Path.Combine(verification, name + ".json"),
JsonSerializer.Serialize(data, options));
string command = args.FirstOrDefault() ?? "all";
try
{
  if (command is "all" or "samples")
  {
    Directory.CreateDirectory(Path.Combine(root, "samples"));
    File.WriteAllBytes(Path.Combine(root, "samples/sample.pdf"), Samples.Pdf());
    File.WriteAllBytes(Path.Combine(root, "samples/sample.epub"), Samples.Epub());
    File.WriteAllBytes(Path.Combine(root, "samples/zip-slip.zip"), Samples.Zip(("../escape.pdf",
Samples.Pdf())));
  }
  foreach (string name in new[] { "standards", "harmonization", "quality", "threats", "bugs" })
    if (command == "all" || command == name)
    {
      string text = File.ReadAllText(Path.Combine(root, "artifacts", name + ".json"));
      File.WriteAllText(Path.Combine(verification, name + ".json"), text);
      var rows = JsonDocument.Parse(text).RootElement;
      string Escape(string s) => "\"" + s.Replace("\"", "\"\"") + "\"";
      var lines = new List<string> { string.Join(';', rows[0].EnumerateObject().Select(x => Escape(x.Name))) };
      lines.AddRange(rows.EnumerateArray().Select(row => string.Join(';', row.EnumerateObject().Select(x
=> Escape(x.Value.ToString())))));
      File.WriteAllLines(Path.Combine(verification, name + ".csv"), lines, new UTF8Encoding(true));
      Console.WriteLine(name + ": строк " + rows.GetArrayLength());
    }
  if (command is "all" or "audit" or "metrics")
  {
    string before = File.ReadAllText(Path.Combine(root, "before/LegacyUploadController.cs"));
    string after = File.ReadAllText(Path.Combine(root, "src/Library.Api/UploadController.cs"));
    string catalog = File.ReadAllText(Path.Combine(root, "src/Library.Api/BooksController.cs"));
    var result = new { before = CodeQuality.Measure(before, "LegacyUploadController"),
      after = CodeQuality.Measure(after, "UploadController"), catalog = CodeQuality.Measure(catalog,
"BooksController"),
      beforeAudit = CodeQuality.Audit(before), afterAudit = CodeQuality.Audit(after), catalogAudit =
CodeQuality.Audit(catalog) };
    Save("analysis", result);Console.WriteLine(JsonSerializer.Serialize(result, options));
  }
```

```
  if (command == "parse-tests")
  {
    string path = args.Length > 1 ? args[1] : Path.Combine(verification, "library-tests.trx");
    var tests = XDocument.Load(path).Descendants().Where(x => x.Name.LocalName == "UnitTestResult")
      .Select(x => new { name = (string?)x.Attribute("testName"), outcome = (string?)x.Attribute("outcome"),
duration = (string?)x.Attribute("duration") }).OrderBy(x => x.name).ToArray();
    if (tests.Length == 0) throw new IOException("Нет результатов TRX");
    var result = new { total = tests.Length, passed = tests.Count(x => x.outcome == "Passed"), failed =
tests.Count(x => x.outcome == "Failed"), skipped = tests.Count(x => x.outcome is not ("Passed" or "Failed")),
tests };
    Save("tests", result);Console.WriteLine($"Всего {result.total}; Passed {result.passed}; Failed
{result.failed}; Skipped {result.skipped}");
    if (result.failed > 0 || result.skipped > 0) Environment.ExitCode = 1;
  }
  if (command == "api-checks")
  {
    using var client = new HttpClient { BaseAddress = new Uri(args.Length > 1 ? args[1] :
"http://127.0.0.1:5352") };
    using var response = await client.PostAsync("/lab/scenarios", null);response.EnsureSuccessStatusCode();
    string text = await response.Content.ReadAsStringAsync();File.WriteAllText(Path.Combine(verification,
"api-scenarios.json"), text);Console.WriteLine(text);
    if (JsonDocument.Parse(text).RootElement.EnumerateArray().Any(x => !
x.GetProperty("passed").GetBoolean())) Environment.ExitCode = 1;
  }
  if (command == "open-ui")
  {
    var url = new Uri(args[1]);using var client = new HttpClient { Timeout = TimeSpan.FromSeconds(2) };bool
ready = false;
    for (int i = 0; i < 30 && !ready; i++)
    {
      try { using var response = await client.GetAsync(new Uri(url, "/health"));ready =
response.IsSuccessStatusCode; }
      catch (HttpRequestException) { } catch (TaskCanceledException) { }
      if (!ready) await Task.Delay(500);
    }
    if (!ready) throw new IOException("Сервер ещё не запущен");
    Process.Start(new ProcessStartInfo(url.AbsoluteUri) { UseShellExecute = true });
  }
}
catch (Exception e) { Console.Error.WriteLine("Операция не выполнена: " +
e.Message);Environment.ExitCode = 1; }
```

src/Library.Core/FilePolicy.cs

```
using System.IO.Compression;
using System.Text;
using System.Xml;
using System.Xml.Linq;
namespace Library.Core;

public sealed record AcceptedFile(string OriginalName, byte[] Bytes, string Extension);
public sealed class RejectedFile(string message, int status = 400) : Exception(message)
{
  public int Status { get; } = status;
}

/// <summary>Проверяет формат и пути до записи недоверенного содержимого.</summary>
public static class FilePolicy
{
  public const int BookLimit = 1024 * 1024;
```

```
public const int InputLimit = 2 * BookLimit;
public const int InflatedLimit = 3 * BookLimit;
public const int EntryLimit = 12;

/// <summary>Проверяет сегменты пути с учётом имён Windows.</summary>
public static bool IsSafeName(string name)
{
   if (string.IsNullOrWhiteSpace(name) || name.Length > 200 ||
      name.Any(c => char.IsControl(c) || c is ':' or '"' or '*' or '?' or '<' or '>' or '|'))
      return false;
   string normalized = name.Replace('\\', '/');
   if (normalized.StartsWith('/')) return false;
   foreach (string part in normalized.Split('/'))
   {
      if (part is "" or "." or ".." || part.EndsWith('.') || part.EndsWith(' ')) return false;
      string stem = part.Split('.')[0].ToUpperInvariant();
      if (new[] { "CON", "PRN", "AUX", "NUL", "COM1", "LPT1" }.Contains(stem)) return false;
      if (stem.Length == 4 && (stem.StartsWith("COM") || stem.StartsWith("LPT")) &&
         stem[3] is >= '1' and <= '9') return false;
   }
   return true;
}

/// <summary>Читает поток с ограничением фактически полученных байтов.</summary>
public static byte[] ReadLimited(Stream stream, int limit)
{
   using var output = new MemoryStream();
   byte[] buffer = new byte[8192];
   int count;
   while ((count = stream.Read(buffer)) != 0)
   {
      if (output.Length + count > limit) throw new RejectedFile("Превышен размер файла", 413);
      output.Write(buffer, 0, count);
   }
   return output.ToArray();
}

/// <summary>Возвращает проверенный набор; сохранение выполняется отдельно.</summary>
public static AcceptedFile[] Validate(string filename, byte[] bytes)
{
   if (!IsSafeName(filename)) throw new RejectedFile("Небезопасное имя файла");
   if (bytes.Length == 0) throw new RejectedFile("Пустой файл");
   if (bytes.Length > InputLimit) throw new RejectedFile("Входной файл больше 2 МиБ", 413);
   if (Path.GetExtension(filename).Equals(".zip", StringComparison.OrdinalIgnoreCase))
      return ValidateZip(bytes);
   return [ValidateBook(filename, bytes)];
}

private static AcceptedFile ValidateBook(string name, byte[] bytes)
{
   if (bytes.Length == 0) throw new RejectedFile("Пустая книга");
   if (bytes.Length > BookLimit) throw new RejectedFile("Книга больше 1 МиБ", 413);
   string extension = Path.GetExtension(name).ToLowerInvariant();
   if (extension == ".pdf")
   {
      if (!bytes.AsSpan().StartsWith("%PDF-"u8) ||
         !Encoding.ASCII.GetString(bytes.AsSpan(Math.Max(0, bytes.Length - 1024))).Contains("%%EOF"))
         throw new RejectedFile("Сигнатура PDF не соответствует расширению");
```

```
    }
    else if (extension == ".epub") ValidateEpub(bytes);
    else throw new RejectedFile("Разрешены PDF, EPUB и ZIP-пакеты");
    return new(name, bytes, extension);
  }

  private static AcceptedFile[] ValidateZip(byte[] bytes)
  {
    try
    {
      using var zip = new ZipArchive(new MemoryStream(bytes));
      if (zip.Entries.Count is 0 or > EntryLimit) throw new RejectedFile("От 1 до 12 записей ZIP");
      var names = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
      var files = new List<AcceptedFile>();
      long total = 0;
      foreach (var entry in zip.Entries)
      {
        if (!IsSafeName(entry.FullName) || !names.Add(entry.FullName.Replace('\\', '/')) ||
          ((entry.ExternalAttributes >> 16) & 0xF000) == 0xA000)
          throw new RejectedFile("Небезопасный путь, ссылка или дубль ZIP");
        if (entry.Length > BookLimit) throw new RejectedFile("Книга больше 1 МиБ", 413);
        if (entry.Length > 0 && (double)entry.Length / Math.Max(1, entry.CompressedLength) > 200)
          throw new RejectedFile("Коэффициент сжатия больше 200");
        using var stream = entry.Open();
        byte[] content = ReadLimited(stream, BookLimit);
        total += content.Length;
        if (total > InflatedLimit) throw new RejectedFile("Объём распаковки больше 3 МиБ", 413);
        files.Add(ValidateBook(entry.FullName, content));
      }
      return files.ToArray();
    }
    catch (InvalidDataException) { throw new RejectedFile("Повреждённый ZIP"); }
  }

  private static void ValidateEpub(byte[] bytes)
  {
    try
    {
      using var zip = new ZipArchive(new MemoryStream(bytes));
      if (zip.Entries.Count > EntryLimit) throw new RejectedFile("Слишком много записей EPUB");
      var names = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
      long total = 0;
      foreach (var e in zip.Entries)
      {
        if (!IsSafeName(e.FullName) || !names.Add(e.FullName) ||
          ((e.ExternalAttributes >> 16) & 0xF000) == 0xA000)
          throw new RejectedFile("Небезопасная структура EPUB");
        using var input = e.Open();
        total += ReadLimited(input, BookLimit).Length;
        if (total > InflatedLimit) throw new RejectedFile("Большой EPUB", 413);
      }
      var mime = zip.GetEntry("mimetype") ?? throw new RejectedFile("Нет mimetype EPUB");
      using var mimeReader = new StreamReader(mime.Open());
      if (mimeReader.ReadToEnd() != "application/epub+zip") throw new RejectedFile("Неверный
mimetype");
      var container = zip.GetEntry("META-INF/container.xml") ?? throw new RejectedFile("Нет
container.xml");
      using var xml = XmlReader.Create(container.Open(), new XmlReaderSettings
```

```
            { DtdProcessing = DtdProcessing.Prohibit, XmlResolver = null, MaxCharactersInDocument = 8192 });
            var doc = XDocument.Load(xml);
            string? package = doc.Descendants().FirstOrDefault(x => x.Name.LocalName == "rootfile")?
                .Attribute("full-path")?.Value;
            if (package is null || !IsSafeName(package) || zip.GetEntry(package) is null)
                throw new RejectedFile("Нет пакета EPUB");
        }
        catch (InvalidDataException) { throw new RejectedFile("Повреждённый EPUB"); }
        catch (XmlException) { throw new RejectedFile("Небезопасный XML EPUB"); }
    }
}
```